

Combinatorial Zero-Shot Prompt Optimization via Ant Colony Optimization

Giammarco Vadicchino

Contributing authors: vadicchinogiammarco@studenti.unimi.it;

Abstract

Large Language Models (LLMs) are widely used for abstractive summarization, yet their performance strongly depends on prompt design, which is often manual and ad hoc. In this work, I formulate prompt optimization as a structured combinatorial search problem and propose an Ant Colony Optimization (ACO) approach to automatically discover effective prompts. Prompts are represented as combinations of functional components and evaluated using relevance metrics (ROUGE and BERTScore) and faithfulness measured via Natural Language Inference. I compare ACO-optimized prompts against human-written prompts and show that the proposed method achieves competitive performance.

1 Introduction

The Introduction section, of referenced text [?] expands on the background of the work (some overlap with the Abstract is acceptable). The introduction should not include subheadings.

Springer Nature does not impose a strict layout as standard however authors are advised to check the individual requirements for the journal they are planning to submit to as there may be journal-level preferences. When preparing your text please also be aware that some stylistic choices are not supported in full text XML (publication version), including coloured font. These will not be replicated in the typeset article if it is accepted.

2 Methodology

2.1 Goal of the Project

This project aims to explore the feasibility of automating prompt design for abstractive text summarization using a meta-heuristic optimization approach. It focuses on the development and evaluation of an Ant Colony Optimization based pipeline for identifying effective prompt configurations for Large Language Models. A key objective is to systematically assess the performance of automatically optimized prompts in comparison with human-crafted prompts, ensuring a fair and reproducible experimental setup. Additionally, the project seeks to examine the stability of the optimized prompts under stochastic generation settings and to evaluate their quality using established lexical, semantic, and faithfulness-oriented metrics.

2.2 Overview of the Approach

The proposed approach formulates prompt engineering for abstractive text summarization as a combinatorial optimization problem. A structured prompt space is defined by decomposing prompts into modular components, each corresponding to a specific instruction or constraint. An Ant Colony Optimization algorithm is then employed to iteratively explore this space, constructing candidate prompts and refining them based on performance feedback obtained from automatic evaluation metrics. The quality of each prompt is assessed by generating multiple summaries with a fixed Large Language Model under controlled stochastic settings, and by aggregating lexical, semantic, and faithfulness-based scores. Through iterative pheromone updates, the optimization process should progressively biases the search toward prompt configurations that yield higher-quality and more stable summaries.

2.3 Formal Definition of the Problem

Let $D = \{d_1, d_2, \dots, d_n\}$ be a set of source document, and let $S = \{s_1, s_1, \dots, s_n\}$ be the corresponding reference summaries. The task of prompt optimization can be formally defined as finding a prompt p^* from a structure prompt research space P that maximizes the expected quality of generated summaries:

$$p^* = \arg \max_{p \in P} \mathbb{E}_{\hat{s} \sim \text{LLM}(d, p)} [Q(\hat{s}, s)] \quad (1)$$

where $\hat{s}$ denotes a summary generated by the LLM given document d and prompt p , and $Q(s, \hat{s})$ is a fitness function. In this project the fitness function aggregates lexical (e.g. ROUGE), semantic (e.g. BERTScore), and faithfulness (e.g. NLI-based) metrics. The objective is to identify prompts that consistently yield high-quality, semantically relevant and factually faithful summaries across the dataset.

2.4 ACO Prompt Optimization

The main challenge of applying Ant Colony Optimization is that the problem needs to be represented as a path-based graph. To accommodate zero-shot structured prompts, I used a Directed Acyclic Graph (DAG). In this representation, a structured prompt—which may include multiple levels of instructions or constraints (e.g., Role Instruction, Style Guidance, Task Instruction)—is decomposed such that each level corresponds to a set of nodes in the graph. To maintain diversity during the search, multiple variants of each node are included, representing alternative ways to phrase or structure the instruction at that level. Each path through the DAG, combining one node from each level, therefore represents a valid candidate prompt, allowing the ACO algorithm to explore a rich and diverse set of prompt configurations. After we build this graph we can apply ACO that we can divide in three phases:

1. **Initialization:** in this phase each edge is assigned an initial pheromone level, some parameters are set such as pheromone influence (α), evaporation rate and pheromone scaling factor (Q).
2. **Path construction:** in this phase each ant starts at the start node of the DAG, then he cross the graph and build a path (that represents a prompt) probabilistically, the probability of choosing the next node depends on pheromone level. When the ants reach the last level of the DAG, the path is serialized into a prompt and the the prompt is ready to be evaluated.
3. **Pheromone Update:** in this phase pheromones evaporate on all edge to avoid saturation. Edges of path found by ants are reinforced according to the chosen strategy:
 - **Classic:** each ant reinforces the edges it traveled, with path with better fitness value leaving more pheromone. This approach spreads reinforcement across all discovered paths, but good (path with high fitness value) paths naturally get a stronger signal.
 - **Elitist:** works like the classic system, but the global path receives an extra boost of pheromones. This accelerates convergence by making the best path more likely to be chosen in future iterations.
 - **Rank-Based:** only the top K paths (the K paths with highest fitness value in the current iteration) contribute to the pheromone update, the higher a path ranks, the more pheromone it deposits.
 - **Max-Min:** only the best path in the current iteration updates pheromones, upper and lower bounds are imposed on the pheromone levels to prevent any edge from becoming too dominant (too much pheromone) or too weak (too less pheromone).
 - **Best-Worst:** combines reinforcement of the best path with the reduction of pheromones on the worst path. By decreasing the attractiveness of poor paths, it reduces the chance they are chosen in the future.

All these phases are repeated for a fixed number of iterations. Through pheromone reinforcement, the ants are progressively guided toward prompt configurations that yield higher fitness values.

3 Experiments Setup and Results

To run the experiments the ACO algorithm were initialized with 4 ants and 4 iterations. Each created prompt was evaluated on the same dataset. The dataset is a list of 16 couples (article, reference summary), that are sampled from a bigger dataset (**INDICARE DATASET**). Each experiment used the same 16 samples. The model that generates the summaries is T5-base (**METTERE REFERENZA MODELLO**), a lightweight encoder-decoder Large Language Model, with approximately 220 million parameters, well suited for abstractive summarization tasks. The model is used in a zero-shot setting, without any task-specific fine-tuning, in order to isolate the effect of prompt optimization. All summaries are generated using fixed decoding parameters across experiments to ensure a fair comparison between different prompt configurations and temperature = 0.3. The metrics that i used in order to evaluate the prompts are:

- **ROUGE**: This metric is obtained from a weighted sum between ROUGE-1 (unigram overlap), ROUGE-2 (bigram overlap), ROUGE-L (Longest Common Subsequence), for each of these metrics i take only the F1 scores, because it balances precision and recall, and the final value is a weighted sum of these metrics. I used it to capture how much explicit content is retrieved from the reference summary.
- **Sentence similarity**: This metric between generated and reference summaries is measured using cosine similarity between sentence embeddings produced by the paraphrase-MiniLM-L6-v2 sentence embedding model. A lightweight model with approximately 22 millions parameters. This metric is used to capture whether the generated summary and the reference summary convey the same meaning. In particular, it is able to account for paraphrasing phenomena in the reference summaries, which are not effectively captured by ROUGE-based metrics.
- **Faithfulness**:

The fitness function that gives a fitness value for each prompt is a weighted sum of the three metrics, where 0.3, 0.2, and 0.5 are the correspondent weights. Note that, due to limited computational resources, the algorithm’s hyper-parameters were not fine-tuned.

4 Discussions and Future Works